

DUETS: A Dataset of Reproducible Pairs of Java Library-Clients

Thomas Durieux , César Soto-Valero , and Benoit Baudry 
KTH Royal Institute of Technology, Stockholm, Sweden
 Email: {tdurieux, cesarsv, baudry}@kth.se

Abstract—Software engineering researchers look for software artifacts to study their characteristics or to evaluate new techniques. In this paper, we introduce DUETS, a new dataset of software libraries and their clients. This dataset can be exploited to gain many different insights, such as API usage, usage inputs, or novel observations about the test suites of clients and libraries. DUETS is meant to support both static and dynamic analysis. This means that the libraries and the clients compile correctly, they are executable and their test suites pass. The dataset is composed of open-source projects that have more than five stars on GitHub. The final dataset contains 395 libraries and 2,874 clients. Additionally, we provide the raw data that we use to create this dataset, such as 34,560 `pom.xml` files or the complete file list from 34,560 projects. This dataset can be used to study how libraries are used by their clients or as a list of software projects that successfully build. The client's test suite can be used as an additional verification step for code transformation techniques that modify the libraries.

Index Terms—Mining software repositories, software reuse, Java, Maven

I. INTRODUCTION

SOFTWARE engineering research requires real software artifacts either to study their properties or to evaluate new techniques. Software datasets have emerged in the community as an effort to standardize and increase the reproducibility of software studies and the comparison between contributions. Each dataset focuses on one specific goal and has specific properties. For example, some datasets focus on the source code of different projects [1], others focus on software that compiles correctly [2], or even focus on specific characteristics such as having known and reproducible bugs [3], [4].

This paper presents a new dataset of software projects: DUETS. Its name reflects the spirit of the library-client relationship. It consists of a collection of Java libraries, which build can be successfully reproduced, and Java clients that use those libraries. DUETS aims to simplify research that focuses on behavioral analysis of Java software. In particular we want to encourage studies that analyze library behavior in a context. The availability of a set of clients for each library supports studies about the actual usage of the library. The dataset can be used both for static analyses and for dynamic analyses by executing the tests of the clients. For that purpose, we take a special care to build a dataset for which we ensure that both the library and the clients have a passing test suite.

This new dataset supports a wide range of usage purposes. In general, many program analyses require a list of compilable and testable software packages, which we provide with

DUETS. The dataset also supports more specific use cases, such as analyzing the API usage by the clients of a particular library [5], or debloating based on dynamic analysis [6], [7]. We provide a framework with the dataset. It can be used, for example, to detect projects that have flaky builds or study the reasons for flakiness or the build [8]. In addition, the reproducible property of the project builds provides a sound ground for empirical studies on how `pom.xml` files (Maven build configuration file) are engineered [9], [10], [11].

We design DUETS to contain a large diversity of projects and focus on the reproducibility of the build. We select only single-module Maven projects to simplify the reproducibility. Multi-module projects tend to increase the complexity and the fragility of the build and make it harder to analyze, study, and instrument. We also build each project three times, as an effort to ensure the reproducibility of the build. The test suite of the project has to pass and only have passing tests.

To summarize the contributions of this paper are the follow:

- DUETS, a dataset of 395 libraries and 2,874 clients. Both the libraries and the clients build successfully with Maven, i.e. all the test pass and a compiled artifact is produced as a result of the build.
- A framework to generate the dataset. It includes scripts for the collection of data and execution of the tests in a sandboxed environment.
- Raw data files from the dataset generation that can be reused for further researches, e.g., 34,560 `pom.xml` files, 33,513 Travis CI configuration files, and 17,403 Dockerfiles.

II. DATA COLLECTION METHODOLOGY

In this section, we describe the methodology that we follow to construct this dataset of open-source Maven Java projects extracted from GitHub. DUETS is composed of two parts: a set of libraries, i.e., Java projects that are declared as a dependency by other Java projects, and a set of clients, i.e., Java projects that use the libraries from the first set. The construction of this dataset is performed in 5 steps. The process is illustrated in Figure 1 and detailed in the following sections.

1) *Collection of Java projects from GitHub*: First, we use the list of Java projects extracted from GitHub by Loriot et al. [12]. The authors queried the GitHub API on June 9th of 2020, to find all the projects that use Java as the primary programming language. The projects found were subsequently



Fig. 1. Overview of the data collection methodology: from a collection of Java projects on GitHub, it produces reproducible sets of libraries and clients.

filtered, discarding those that have less than 5 stars. This initial dataset includes 147,991 Java projects.

2) *Identification of single-module projects*: Second, we select the subset of single-module projects among the 147,991 Java projects. We choose single-module projects to have a clear mapping between client and library and have more reliable build reproduction.

We download the complete list of files for each project using the GitHub API. We consider that a project is a single-module project when it has a single Maven build configuration file, i.e., `pom.xml`. We exclude the `pom.xml` files that are in `resource` folders and `test` folders. At the end of this step, we keep 34,560 (23.4%) single-module Maven projects. The list of all the files is also part of our dataset and is available in the repository of the dataset.

3) *Identification of libraries and clients*: In the third step, we analyze each `pom.xml` from the 34,560 projects. During the analysis, we first extract the `groupId` and `artifactId` qualifiers of each project. This pair of ids is used by Maven to identify a project. In the case where two projects declare the same pair of `groupId` and `artifactId`, we select the project that has the largest number of stars on GitHub. Second, we map the `groupId` and `artifactId` to the dependency declared in the `pom.xml`. At the end of this step, we obtain a list of projects that are used as dependency, i.e., libraries and a list of clients that use the libraries. During this step, we ignore the projects that do not declare JUnit as a testing framework, and we exclude the projects that do not declare a fixed release, e.g., `LAST-RELEASE`, `SNAPSHOT`. After this third step, we identify 155 (0.4%) libraries, and 25,557 (73.9%) clients that use 2,103 versions of the libraries.

4) *Identification of commits for each library release*: The purpose of the fourth step is to identify the commit SHA identifier that determines each version of the library, i.e., the commit change in the `pom.xml` that assigns a new version. For example, the version 3.4 of the library `commons-net` is defined in the commit SHA 74a2282b7e4c6905581f4f1b5a2ec412310cd5e7. To perform this task, we download all revisions of the Maven build configuration files since their creation. Then, we analyze the Maven build configuration files, and identify which commit declares a specific release of the library. We successfully identify the commit for 1,026/2,103 (48.8%) versions for 143/155 (92.3%) libraries. 16,964/25,557 (66.4%) clients have been mapped to a specific commit of one of their dependencies.

5) *Execution of the tests*: As the fifth and last step, we execute three times the test suite of all library versions and all clients, as a sanity check to filter out libraries with flaky

tests or projects that cannot be built. We keep the libraries and clients that have at least one test and have all tests passing: 94/143 (65.7%) libraries, 395/1,026 (38.5%) library versions, and 2,874/16,964 (16.9%) clients passed this verification. From this point, we consider each library version as a unique library for clarification purpose.

III. DESCRIPTION OF THE DATASET

Table I summarizes the descriptive statistics of the dataset. The number of lines of code (#LOC) and the coverage are computed with JaCoCo. DUETS includes 94 different libraries, with a total of 395 versions, as well as 2,874 clients. Those libraries and clients are maintained by 1,669 different GitHub organizations. The libraries include 713,932 test cases that cover 80.83% of the 10,831,394 LOC. The libraries have a median maintenance time of 9.49 years from 623 commits created by 29 contributors. The clients have 211,116 test cases that cover 20.24% of the 140,910,102 LOC. The clients have a median maintenance time of 2.65 years from 86 commits created by 4 contributors. The dataset and the scripts to generate the dataset are publicly available in our experiment repository: <https://github.com/castor-software/Duets>.

A. Dataset format

DUETS is available on GitHub and is composed of a JSON file.¹ An excerpt of this JSON file is presented in the README of our repository. It contains the repository name, the SHA of the commit, the `groupId`, `artifactId`, the list of clients for each version of the library, and a list of commits that defines the different releases of the library. DUETS also contains the logs corresponding to the test execution for each version of the libraries and for each client. As previously mentioned, we executed three times the tests to increase the likelihood of the reproducibility of the dataset.

In addition to the dataset itself, we include all scripts that generate the dataset. Those scripts can be used to reproduce the same dataset, to create a similar dataset for a different language or reusing too mine Github.

B. Execution framework

In addition to the JSON file, we provide a Docker image that contains our execution framework. This framework adds an abstraction on top of Git repositories. It automatizes the cloning, checkout, execution of the test, and parsing the test results without requiring to specify and

¹<https://github.com/castor-software/Duets/tree/master/dataset/dataset-info.json>

TABLE I
DESCRIPTIVE STATISTICS OF THE STUDIED LIBRARIES AND THEIR ASSOCIATED CLIENTS

		Min	1st Qu.	Mean	3rd Qu.	Max	Avg.	Total
395 Libraries	# Line of code	132	5,439.5	17,935.5	47,866.0	341,429	35,629.6	10,831,394
	# Contributors	1	14.25	29	58.5	286	48.79	4,586
	# Years of activity	0.35	6.25	9.49	15.21	22.35	10.32	N.A
	# Commits	2	217.25	623	1,538.75	16,570	1,283.46	120,645
	# Stars	5	72.75	266.5	701.5	21,865	1,457.82	137,035
	# Tests	1	139.8	378.0	1,108.2	24,946	1,830.6	713,932
	Coverage	0.1 %	61.7 %	80.8 %	89.8 %	100.0 %	73.7 %	N.A
2,874 Clients	# Line of code	0	3,130.0	9,170.0	58,990.0	4,531,710	72,897.1	140,910,102
	# Contributors	1	2	4	11	286	10.98	31,493
	# Years of activity	0	0.94	2.65	4.92	23.80	3.35	N.A
	# Commits	1	27	86	261	18,594	370.26	1,062,286
	# Stars	5	7	14	40	21,865	118.96	341,287
	# Tests	1	4.5	20.0	74.0	11,415	107.7	211,116
	Coverage	<0.1 %	2.1 %	20.24 %	57.7 %	100.0 %	31.4 %	N.A

additional information than the URL of the repository. All those tasks are simplified into the following command line:
`docker run --rm -v 'pwd':/results castorsoftware/duets:latest compile --repository https://github.com/radsz/jacop --commit 8f09fd977a.` This framework can be extended to perform additional tasks, we provide an API that allows to execute the main Maven tasks and to manipulate the pom.xml files easily (e.g., to add or remove plugins). We extended the framework to perform static analysis and to collect the test suite coverage. Those examples are provided in the DUETS repository as guidelines.

IV. DATASET USAGE

A. Pairs of libraries and clients

The clients in DUETS can be used to identify APIs usage patterns between different clients [5], or to explore how the API evolution of the libraries affects their clients [13]. For example, Figure 2 shows a weighted bipartite graph of the relationship between 10 clients (on the left) and the packages belonging to one library, Apache Commons Codec (on the right). The width of the edges between a client and a package represents the number of classes in the package that are executed (either directly called or invoked internally in the library) when running the clients' tests. This type of figure allows to visualize the parts of the library that are more used by its clients, which is valuable information for both the library users and the library maintainers.

The test-suite of the clients can also be used as further validation of modifications performed on the library. This is the type of usage that we leverage in our recent work [6], where we debloat libraries and verify that the compilation and the execution of their clients' tests are not affected. Hence, having information regarding library usage by clients is useful for validating program transformation since it provides dynamic data that helps to overcome the limitations of static analysis in Java.

Another potential usage of the clients of the library is to use the client tests to generate tests for the libraries or to verify that changes in the libraries do not break the clients. This

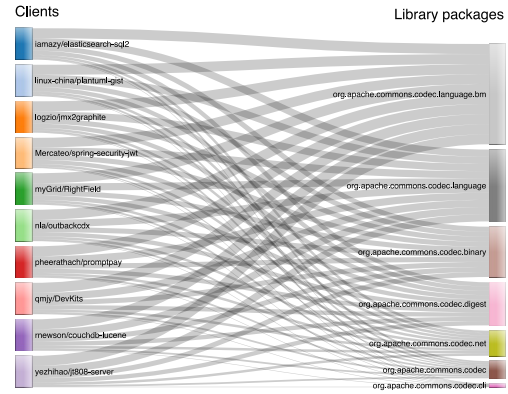


Fig. 2. Sankey diagram representing the number of classes in the packages of the library Apache Commons Codec (v. 1.12) that are used by 10 clients

idea is developed in a recent study performed by Chen and colleagues [14].

DUETS allows to compare the characteristics of libraries with respect to other software artifacts. For example, during our data collection, we observed that libraries have much more tests and have a higher test coverage than the clients (see Table I). A detailed analysis of the test part of the dataset could highlight differences between libraries and other types of applications.

Finally, DUETS can be used to compare the coverage of the library with its own test suite and the test suites of the clients, to identify the intersection and difference between the two test suites, similar to the work of Wang et al. [15] that checks the similarity between test and production behavior.

B. Buildable and testable Java projects

If the relations between the clients and the libraries are not required for a specific evaluation. DUETS can be used as a list of projects that successfully build and have a passing test suite. This can be used as a dataset for dynamic analysis such as identifying API usage based on the client or libraries test suites. This dataset contains a large diversity of projects, large and small, from different fields.

C. Build results of the projects

During the creation of DUETS, we verified that 7,293 projects are reproducible and have only passing tests (see Section II-5). We saved the test results of those executions. This data can be used to identify projects with failing tests, flaky tests or flaky builds. Identifying flaky builds is a hard task. This data could have simplified the work of studies like [16], [17], [18]. Table II presents some metrics of our reproduction attempts. We identify 221 (3.0%) projects that have flaky tests, 1,009 (13.8%) projects with at least one failing test case.

TABLE II
REPRODUCIBILITY METRICS REPORT

Metrics	Value
# Reproduction attempts	7,293
# Buildable projects	3,642 (49.9%) ■
# Unbuildable projects	3,418 (46.9%) ■
# Failing-test builds	1,009 (13.8%) ■
# Flaky builds	221 (3.0%) ■
# Timeout	12 (0.2%) ■

D. List of files of Java projects

We downloaded the complete list of files for 34,560 Java projects from GitHub. This list of files can be used to identify the usage of specific technologies such as Docker, continuous integration, build management systems, or investigating the adoption of some development practice such as including binaries in the repositories. Table III shows the rate of occurrence of these particular files in DUETS. This data could be used for study like the one of Cito et al. [19].

TABLE III
OCCURRENCE OF JAVA FILES AND CONFIGURATION FILES IN THE DATASET OF FILE LISTS

Metrics	Values
# Files	71,768,708
# Java files	21,519,119 (30.0%) ■
# pom.xml files	363,220 (0.5%) ■
# Gradle files	229,690 (0.3%) ■
# Travis files	33,513 (<0.1%) ■
# GitHub Workflow files	33,513 (<0.1%) ■
# Dockerfiles	17,403 (<0.1%) ■

E. Analysis of pom.xml files

DUETS contains 34,560 pom.xml files. Those files can be used to analyze the common usage of pom.xml in open source repositories. This dataset of pom.xml files has several advantages compared to a dataset of pom.xml created directly from Maven Central. The pom.xml in DUETS are directly associated with a Git repository and therefore additional information is available, such as the source code, the history of the project, or issues. This provides a solid starting point to analyze the co-evolution of build files and other artifacts [20].

V. RELATED WORK

Building software is a complex task. Indeed, Kerzazi et al. [21] observe that 17.9% of CI builds in an industrial web application are failing during a period of 6 months. Durieux et al. [18] shows that only 70% of the builds are passing on

Travis CI. Reproducing builds is even an harder task. Sulir et al. [22] show that 40% of the builds in their dataset are not reproducible. Almost 40% of the build problems are related to missing dependencies, followed by compilation errors in 22% of the cases. Gkortzis et al. [23] observe a very similar build failure rate (33.7%), with the same causes. Neitsch et al. [24] analyze the build systems of 5 open-source multi-language Ubuntu packages. They observe that 4 of the 5 packages cannot build or be rebuilt. They find that many build problems can be addressed, and note that build quality is rarely the subject of research, even though it is an important part of maintaining and reusing software.

The software engineering research community came up with several benchmarks of Java projects that focus on reproducibility of the build. Sulir et al. [22] attempted to build 7,264 Java projects from GitHub, from which around 60% of the builds succeeded. Martins et al. [2] presented in 2018 a dataset that follows the same idea but with 50,000 compilable and compiled Java projects. Dacapo by Blackburn et al. [25] consists of a set of open source, real world applications with non-trivial memory loads. The difference between those datasets and DUETS is that we constructed a up-to-date benchmark with recent and diverse projects, we also focus on Maven that have a test suite and all tests are passing.

There are several datasets of buggy Java programs that generally also come with the non-buggy version of the program, such as [3], [4], [26], [27], [28]. Datasets that only focus on source code also exist, such as Boa, a dataset of queryable Java AST presented by Dyer et al. [29]. Spinellis et al. [30] focus on identifying duplicated repositories on GitHub.

The closest work that focuses on studying libraries and their clients is the work from Leuenberger et al. [31]. They analyze the binaries of artifacts in Maven Central to identify API clients. In contrast, we focus on the projects' source code and to allow to build and test the software where Leuenberger et al. are interested to mine the API usage of compiled projects.

The major difference between all those datasets and DUETS is that we focus on pairs of libraries and clients. To our knowledge, this has never been done.

VI. CONCLUSION

In this paper, we presented DUETS, a dataset of 395 libraries and 2,874 clients extracted from open-source projects on GitHub. DUETS aims to simplify studies that rely on dynamic and static analysis of libraries' usage in the Java ecosystem. In our previous work, we have used this dataset to study the impact of debloatig libraries on their clients. However, DUETS also provides a fertile ground for other types of empirical studies, such as those that analyze the impact of API changes on library clients. Alongside the dataset itself, we provide a framework that aims to facilitate the data mining. Both the dataset and the necessary tools to reproduce it are open-source and publicly available online. We also provide the raw data that we use to generate DUETS, including 34,560 pom.xml files and the complete file list of 34,560 Java projects.

ACKNOWLEDGMENT

This work is partially supported by the Wallenberg AI, Autonomous Systems, and Software Program (WASP) funded by Knut and Alice Wallenberg Foundation and by the TrustFull project funded by the Swedish Foundation for Strategic Research.

REFERENCES

- [1] A. Pietri, D. Spinellis, and S. Zacchiroli, "The software heritage graph dataset: Public software development under one roof," in *Proceedings of the 16th International Conference on Mining Software Repositories, MSR '19*, p. 138–142, IEEE Press, 2019.
- [2] P. Martins, R. Achar, and C. V. Lopes, "50k-c: A dataset of compilable, and compiled, java projects," in *2018 IEEE/ACM 15th International Conference on Mining Software Repositories (MSR)*, pp. 1–5, IEEE, 2018.
- [3] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A Database of existing faults to enable controlled testing studies for Java programs," in *ISSTA 2014, Proceedings of the 2014 International Symposium on Software Testing and Analysis*, (San Jose, CA, USA), pp. 437–440, July 2014. Tool demo.
- [4] F. Madeiral, S. Urii, M. Maia, and M. Monperrus, "Bears: An extensible java bug benchmark for automatic program repair studies," in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 468–478, 2019.
- [5] M. A. Saied, O. Benomar, H. Abdeen, and H. Sahraoui, "Mining multi-level api usage patterns," in *2015 IEEE 22nd international conference on software analysis, evolution, and reengineering (SANER)*, pp. 23–32, IEEE, 2015.
- [6] C. Soto-Valero, T. Durieux, N. Harrand, and B. Baudry, "Trace-based Deblat for Java Bytecode," *arXiv e-prints*, p. arXiv:2008.08401, Aug. 2020.
- [7] B. R. Bruce, T. Zhang, J. Arora, G. H. Xu, and M. Kim, "Jshrink: In-depth investigation into debloating modern java applications," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020*, (New York, NY, USA), p. 135–146, Association for Computing Machinery, 2020.
- [8] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?," in *Proceedings of the 17th International Conference on Mining Software Repositories, MSR '20*, (New York, NY, USA), p. 492–502, Association for Computing Machinery, 2020.
- [9] Y. Wang, M. Wen, Z. Liu, R. Wu, R. Wang, B. Yang, H. Yu, Z. Zhu, and S.-C. Cheung, "Do the dependency conflicts in my project matter?," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 319–330, 2018.
- [10] S. McIntosh, B. Adams, and A. E. Hassan, "The evolution of java build systems," *Empir. Softw. Eng.*, vol. 17, no. 4-5, pp. 578–608, 2012.
- [11] C. Soto-Valero, N. Harrand, M. Monperrus, and B. Baudry, "A Comprehensive Study of Bloated Dependencies in the Maven Ecosystem," *arXiv e-prints*, p. arXiv:2001.07808, Jan. 2020.
- [12] B. Lorient, F. Madeiral, and M. Monperrus, "Styler: Learning formatting conventions to repair checkstyle errors," *arXiv preprint arXiv:1904.01754*, 2019.
- [13] A. M. Eilertsen and A. H. Bagge, "Exploring api: Client co-evolution," in *Proceedings of the 2nd International Workshop on API Usage and Evolution*, pp. 10–13, 2018.
- [14] L. Chen, F. Hassan, X. Wang, and L. Zhang, "Taming behavioral backward incompatibilities via cross-project testing and analysis," in *IEEE/ACM International Conference on Software Engineering*, 2020.
- [15] Q. Wang, Y. Brun, and A. Orso, "Behavioral execution comparison: Are tests representative of field behavior?," in *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pp. 321–332, IEEE, 2017.
- [16] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, (New York, NY, USA), pp. 643–653, ACM, 2014.
- [17] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2019*, (New York, NY, USA), p. 830–840, Association for Computing Machinery, 2019.
- [18] T. Durieux, C. Le Goues, M. Hilton, and R. Abreu, "Empirical study of restarted and flaky builds on travis ci," in *Proceedings of the 17th International Conference on Mining Software Repositories, MSR '20*, (New York, NY, USA), p. 254–264, Association for Computing Machinery, 2020.
- [19] J. Cito, G. Schermann, J. E. Wittern, P. Leitner, S. Zumberi, and H. C. Gall, "An empirical analysis of the docker container ecosystem on github," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pp. 323–333, IEEE, 2017.
- [20] S. McIntosh, B. Adams, M. Nagappan, and A. E. Hassan, "Mining co-change information to understand when build changes are necessary," in *2014 IEEE International Conference on Software Maintenance and Evolution*, pp. 241–250, IEEE, 2014.
- [21] N. Kerzazi, F. Khomh, and B. Adams, "Why do automated builds break? an empirical study," in *2014 IEEE International Conference on Software Maintenance and Evolution*, pp. 41–50, IEEE, 2014.
- [22] M. Sulír and J. Porubán, "A quantitative study of java software build-ability," in *Proceedings of the 7th International Workshop on Evaluation and Usability of Programming Languages and Tools*, pp. 17–25, 2016.
- [23] A. Gkortzis, D. Feitosa, and D. Spinellis, "Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities," *Journal of Systems and Software*, vol. 172, p. 110653, 2020.
- [24] A. Neitsch, K. Wong, and M. W. Godfrey, "Build system issues in multilanguage software," in *2012 28th IEEE International Conference on Software Maintenance (ICSM)*, pp. 140–149, IEEE, 2012.
- [25] S. M. Blackburn, R. Garner, C. Hoffmann, A. M. Khang, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, M. Hirzel, A. Hosking, M. Jump, H. Lee, J. E. B. Moss, A. Phansalkar, D. Stefanović, T. VanDrunen, D. von Dincklage, and B. Wiedermann, "The dacapo benchmarks: Java benchmarking development and analysis," *SIGPLAN Not.*, vol. 41, p. 169–190, Oct. 2006.
- [26] D. A. Tomassi, N. Dmeiri, Y. Wang, A. Bhowmick, Y.-C. Liu, P. T. Devanbu, B. Vasilescu, and C. Rubio-González, "Bugswarm: mining and continuously growing a dataset of reproducible failures and fixes," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pp. 339–349, IEEE, 2019.
- [27] R. K. Saha, Y. Lyu, W. Lam, H. Yoshida, and M. R. Prasad, "Bugs. jar: a large-scale, diverse dataset of real-world java bugs," in *Proceedings of the 15th International Conference on Mining Software Repositories*, pp. 10–13, 2018.
- [28] T. Durieux and M. Monperrus, "Introclassjava: A benchmark of 297 small and buggy java programs," 2016.
- [29] R. Dyer, H. A. Nguyen, H. Rajan, and T. N. Nguyen, "Boa: A language and infrastructure for analyzing ultra-large-scale software repositories," in *2013 35th International Conference on Software Engineering (ICSE)*, pp. 422–431, IEEE, 2013.
- [30] D. Spinellis, Z. Kotti, and A. Mockus, "A dataset for github repository deduplication," in *Proceedings of the 17th International Conference on Mining Software Repositories*, pp. 523–527, 2020.
- [31] M. Leuenberger, H. Osman, M. Ghafari, and O. Nierstrasz, "Kowalski: Collecting api clients in easy mode," in *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 653–657, IEEE, 2017.